11 JANUARY 2026

LIVERPOOL HOPE UNIVERSITY
1844

HIGH PERFORMANCE COMPUTING

EVALUATING MULTI-PROCESS PARALLELISM FOR NUMERICAL SIMULATION OF THE LORENZ SYSTEM USING PYTHON

KEVIN PRAJAPATI
24014410
MSc Data Science

Abstract:

This project looks at how multi-process work can make numerical simulations of the Lorenz system run faster in Python. The Lorenz system, given by three nonlinear ordinary differential equations, is solved with the usual fourth-order Runge–Kutta (RK4) method. Fixed values are chosen for σ and β, while ρ is changed to create many independent systems. Two versions are tested: a sequential version that runs all systems in one process, and a parallel version that uses Python's multiprocessing module with a reusable process pool to share simulations across the CPU cores. Performance is checked by timing the runs, finding the speedup, and confirming that the results from both versions match. The results show that treating each Lorenz system as an independent task can lower runtime when the number of systems grows, while keeping accuracy. This underlines how task sharing and process reuse can improve overall efficiency in large simulations.

Table of Contents

1. Introduction: 3
2. Methodology and Parallelization Strategy: 4
2.1 Lorenz System and Choice of RK4: 4
2.2 Sequential Implementation: 4
2.3 Parallel Implementation with Multiprocessing: 5
2.4 Process Reuse Strategy: 5
2.5 Performance Measurement: 5
3. Implementation Details: 6
3.1 Code Structure: 6
3.2 Key Design Choices: 7
3.3 Results Collection: 7
3.4 Visualization of Performance: 8
4. Results and Analysis: 9
4.1 Runtime Comparisons: 9
4.2 Speedup Values for 2 and 4 Processes: 9
4.3 Runtime Scaling Plot: 10
4.4 Summary Table of Results: 10
4.5 Accuracy Verification: 10
4.6 Interpretation of Scaling Behaviour: 11
5. Performance Insights and Future Directions: 11
5.1 Strengths of the Approach: 11
5.2 Limitations Encountered: 11
5.3 Suggested Improvements: 11
6. Conclusion: 12
References: 12

1. Introduction:

Parallel computing is an important method in modern scientific work. It gives a practical way to speed up simulations and data analysis that would otherwise take too long. Many scientific problems have work that can be split into tasks, so they are well suited to running in parallel on processors or cores. This project studies this idea by using multi-process parallelism for the numerical solution of the Lorenz system, a classic model of chaotic motion described by three linked nonlinear ordinary differential equations. The Lorenz system is used in computational science because it is sensitive to small changes in values and parameters, so it is a useful test for checking numerical and parallel methods.

In this work, a solver for the Lorenz system is written in Python with the standard fourth-order Runge–Kutta (RK4) method. The parameters σ and β stay fixed, while ρ changes between runs to create different systems. Each system is solved over the same time range with a fixed time step, which keeps the tests fair and consistent. The main difficulty appears when many systems must be simulated, because this produces a workload that is embarrassingly parallel: each system can be solved on its own without communication between tasks. This feature makes the Lorenz system a good choice for showing the gains from parallel computing.

Two implementations are built and compared. The sequential version runs all simulations one after another inside a single Python process, while the parallel version spreads the tasks across several CPU cores with Python's multiprocessing module. A reusable process pool is used to cut overhead and to follow common high-performance computing practice, where resources are started once and kept active during the job. Performance is tested by measuring runtime for different batch sizes, from 16 to 256 systems, and by trying setups with two and four worker processes. The speedup over the sequential baseline is calculated, and the match between sequential and parallel results is checked to make sure they stay accurate. The results give strong evidence that multi-process parallel work can greatly reduce runtime for large workloads while keeping the solutions correct. This study shows the efficiency and reliability of parallel task sharing in scientific computing.

2. Methodology and Parallelization Strategy:

2.1 Lorenz System and Choice of RK4:

The Lorenz system is a set of three linked nonlinear ordinary differential equations that describe chaotic motion in dynamical systems. It is controlled by the parameters σ, ρ, and β, and gives paths that are sensitive to initial conditions, often moving apart quickly with small changes. Because of this sensitivity, a stable and accurate numerical method is needed for solving to keep results meaningful. In this project, the standard fourth-order Runge–Kutta (RK4) method was chosen to solve the system. RK4 is widely used in scientific computing because it balances accuracy and speed, giving reliable results with a fixed time step while avoiding the instability that can arise with simpler methods such as Euler's scheme. (Yadav, Baig and Mehtre, 2022) Using RK4 makes sure that all simulations are consistent, repeatable, and comparable across different runs, which is important for checks. (Utkarsh et al., 2024)

```
def lorenz_rhs(x, y, z, sigma, rho, beta):
    dx = sigma * (y - x)
    dy = x * (rho - z) - y
    dz = x * y - beta * z
    return dx, dy, dz
```

2.2 Sequential Implementation:

The sequential version of the solver runs all Lorenz systems one after another inside a single Python process, with no parallel work at all. Each system is given fixed values for σ and β, while ρ is changed in steps to create many separate tasks for computation. For every system, the RK4 method solves the equations over the same time range with a fixed step size, which keeps the comparison fair. This version gives a baseline measure of runtime and accuracy, which the parallel version can be checked against in detail.

```
def run_sequential(task_list):
    start_time = time.perf_counter()
    results = []
    for args in task_list:
        results.append(run_one_system(args))
    end_time = time.perf_counter()
    return results, end_time - start_time
```

2.3 Parallel Implementation with Multiprocessing:

The parallel version handles each Lorenz system as a separate task and sends them across several CPU cores using Python's multiprocessing module to work faster. A reusable process pool is made when the program starts, and tasks are given to the pool to run, so extra overhead is avoided. (Aziz et al., 2021) This choice removes the cost of very often starting and stopping processes, which can change performance results. To share the work in a good way, tasks are split into chunks, with chunk size chosen by the number of processes and total tasks. This gives better scheduling, cuts waiting time for workers, and makes better use of the cores. By using the embarrassingly parallel form of the problem, the parallel version greatly lowers runtime and shows that multiprocessing can scale well. (Utkarsh et al., 2024)

```
def run_parallel_with_pool(task_list, pool, process_count):
    start_time = time.perf_counter()
    task_len = len(task_list)
    if task_len == 0:
        return [], 0.0
    chunk_size = max(1, task_len // (process_count * 4))
    results = pool.map(run_one_system, task_list, chunksize=chunk_size)
    end_time = time.perf_counter()
    return results, end_time - start_time
```

2.4 Process Reuse Strategy:

A key design choice was to reuse process pools instead of creating new ones for each batch of simulations, which would be wasteful. Creating processes in Python has overhead, such as memory setup, inter-process communication, and time spent synchronizing. Reusing pools removes this overhead and matches common high-performance computing practice, where resources are set up once and kept running for the whole job to keep throughput high. (LabEx) This approach makes sure that performance tests focus on how the simulations scale in computation, not on the cost of handling processes, giving a clearer view of parallel efficiency. It also keeps the code simpler and more consistent, in line with how HPC systems are run. (Arjona, Finol and García López, 2023)

2.5 Performance Measurement:

Performance was checked using three measures: runtime, speedup, and accuracy, each giving a different view of efficiency. Runtime was taken with Python's *time.perf_counter()* to get exact times for both sequential and parallel runs with different numbers of systems. Speedup was found as the ratio of sequential runtime to parallel runtime, giving a simple measure of gain

and scaling. Accuracy was tested by comparing the final states from sequential and parallel simulations, and the maximum difference across all systems was saved to check consistency. This showed that parallel running did not add numerical errors and that results stayed close to the sequential case, while also proving the performance gains and parallel efficiency clearly in practice.

3. Implementation Details:

3.1 Code Structure:

The project is split into clear functions, each taking care of one part of the work. The mathematical model, numerical method, and ways of running the code are kept separate to make the program easy to read and change. Some functions set up the Lorenz equations, use the RK4 method, run simulations in order or in parallel, and test the accuracy. The main function runs the tests, saves runtimes and speedups, and makes plots and tables. This shows parallel computing.

- **lorenz_rhs:** This function gives the three linked nonlinear equations of the Lorenz system. It returns the instant rates of change of x, y, and z from the values of σ, ρ, and β for use in numerical integration.
- **integrate_lorenz:** This function uses the fourth-order Runge–Kutta (RK4) method to move the Lorenz system over a fixed time range. It repeats the RK4 steps with a constant step size to keep the solution stable and accurate for the chaotic system.
- **run_one_system:** This function is a wrapper that runs one Lorenz system for given parameters and starting values. It makes task handling easier by turning the integration into one callable job for both sequential and parallel runs.
- **run_sequential:** This function runs all Lorenz systems one by one in a single Python process. It measures the total runtime with time.perf_counter(), giving a baseline to compare with the parallel version.
- **run_parallel_with_pool:** This function spreads tasks over several CPU cores using Python's multiprocessing pool. It chooses a chunk size to share the work and measures the runtime, showing how parallel execution can cut compute time.
- **max_difference:** This function checks results from sequential and parallel runs to confirm numerical accuracy. It finds the largest Euclidean distance between final states, making sure parallel work does not add error.

- **main:** This function runs the full study by building the task list, calling the sequential and parallel functions, and saving runtimes and speedups. It also creates plots and tables, giving numerical and visual proof of the performance gains.

3.2 Key Design Choices:

Several design choices were made to ensure consistency and fairness in performance testing. The parameters σ and β are fixed at 10.0 and 8/3 respectively, while ρ is varied across runs to generate multiple independent Lorenz systems. Each system is integrated over the same time interval with a fixed step size (dt = 0.01), ensuring that all simulations follow identical numerical schemes. This setup highlights the embarrassingly parallel nature of the workload, since each Lorenz system can be solved independently without communication between tasks.

```
sigma_value = 10.0
   beta_value = 8.0 / 3.0
   base_rho = 20.0
   t_start = 0.0
   t_end = 80.0
   dt = 0.01
   initial_state = (1.0, 1.0, 1.0)
```

3.3 Results Collection:

Performance results were gathered by timing runs and computing speedup for different system counts and process settings. Runtime was taken with Python's *time.perf_counter()* to record accurate execution times for both sequential and parallel runs. Speedup was then found as the ratio of sequential runtime to parallel runtime, showing how much faster the parallel version was. These numbers were stored in lists (*seq_times*, *par_times_2*, *par_times_4*, *speedups_2*, *speedups_4*) so they could be shown in plots and summed up in tables.

```
seq_results, seq_time = run_sequential(tasks)
            seq_times.append(seq_time)
            print(f"\nsystems={system_count}, sequential
time={seq_time:.3f}s")

            par_time_for_2 = None
            par_time_for_4 = None
            speedup_for_2 = None
            speedup_for_4 = None

            for procs in process_options:
                pool = pools[procs]
                par_results, par_time = run_parallel_with_pool(
                    tasks, pool, procs
```

```
                )
                speedup = seq_time / par_time
                diff = max_difference(seq_results, par_results)
                print(
                    f"  processes={procs}, parallel time={par_time:.3f}s, "
                    f"speedup={speedup:.2f}x, max_diff={diff:.3e}"
                )
                if procs == 2:
                    par_time_for_2 = par_time
                    speedup_for_2 = speedup
                elif procs == 4:
                    par_time_for_4 = par_time
                    speedup_for_4 = speedup

            par_times_2.append(par_time_for_2)
            par_times_4.append(par_time_for_4)
            speedups_2.append(speedup_for_2)
            speedups_4.append(speedup_for_4)
```

3.4 Visualization of Performance:

Performance results were gathered by timing runs and computing speedup for different system counts and process settings. Runtime was taken with Python's *time.perf_counter()* to record accurate execution times for both sequential and parallel runs. Speedup was then found as the ratio of sequential runtime to parallel runtime, showing how much faster the parallel version was. These numbers were stored in lists (*seq_times*, *par_times_2*, *par_times_4*, *speedups_2*, *speedups_4*) so they could be shown in plots and summed up in tables.

```
  plt.figure()
    plt.plot(system_counts, seq_times, marker="o", label="Sequential")
    plt.plot(system_counts, par_times_2, marker="o", label="Parallel (2
processes)")
    plt.plot(system_counts, par_times_4, marker="o", label="Parallel (4
processes)")
    plt.xlabel("Number of systems")
    plt.ylabel("Runtime (s)")
    plt.title("Sequential vs Parallel runtimes")
    plt.legend()
    plt.grid(True)
    plt.savefig("lorenz_seq_vs_par.png", dpi=300, bbox_inches="tight")
    plt.show()

    print("\nSummary table (times in seconds):")
    print("systems  seq_time   par2_time  speedup2   par4_time  speedup4")
    for i, systems in enumerate(system_counts):
        print(
```

```
            f"{systems:7d}  "
            f"{seq_times[i]:8.3f}  "
            f"{par_times_2[i]:9.3f}  "
            f"{speedups_2[i]:8.2f}x  "
            f"{par_times_4[i]:9.3f}  "
            f"{speedups_4[i]:8.2f}x"
        )
```

4. Results and Analysis:

4.1 Runtime Comparisons:

The performance of the sequential and parallel versions was first checked using runtime. For each number of systems, the sequential version ran all Lorenz systems one after another, while the parallel version spread the tasks across several processes. As expected, sequential runtimes grew steadily as the number of systems increased, while parallel runtimes stayed lower, showing the benefit of multiprocessing. This comparison gives the basic proof of speedup gained through parallel execution.

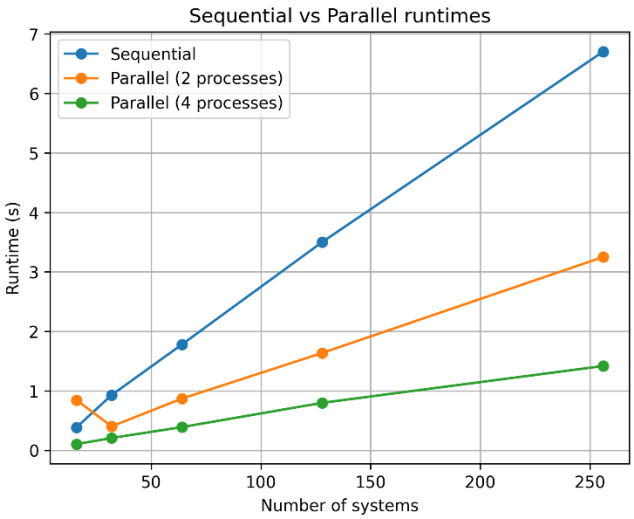
```
PS D:\Semester 2\High Performance Computing> python -u "d:\Semester 2\High Performance Computing\Lorenz.py"

systems=16, sequential time=0.387s
  processes=2, parallel time=0.843s, speedup=0.46x, max_diff=0.000e+00
  processes=4, parallel time=0.103s, speedup=3.74x, max_diff=0.000e+00

systems=32, sequential time=0.929s
  processes=2, parallel time=0.402s, speedup=2.31x, max_diff=0.000e+00
  processes=4, parallel time=0.207s, speedup=4.49x, max_diff=0.000e+00

systems=64, sequential time=1.779s
  processes=2, parallel time=0.869s, speedup=2.05x, max_diff=0.000e+00
  processes=4, parallel time=0.390s, speedup=4.56x, max_diff=0.000e+00

systems=128, sequential time=3.502s
  processes=2, parallel time=1.637s, speedup=2.14x, max_diff=0.000e+00
  processes=4, parallel time=0.798s, speedup=4.39x, max_diff=0.000e+00

systems=256, sequential time=6.703s
  processes=2, parallel time=3.250s, speedup=2.06x, max_diff=0.000e+00
  processes=4, parallel time=1.417s, speedup=4.73x, max_diff=0.000e+00
```

4.2 Speedup Values for 2 and 4 Processes:

Speedup was worked out as the ratio of sequential runtime to parallel runtime. With two processes, the speedup values showed moderate gain, while four processes gave stronger improvement. Although speedup did not grow in a perfectly straight line with more processes, the results still showed that parallel work cut runtime a lot. These numbers still show the efficiency gains possible even with few cores.

4.3 Runtime Scaling Plot:

A plot was made to show how runtime changes when the number of systems increases. The figure compared sequential execution with parallel execution using two processes and four processes. The sequential line went up quickly as the workload grew, while the parallel line rose more slowly. This picture strongly supports the numbers and clearly shows good parallel efficiency overall in this case.



4.4 Summary Table of Results:

A summary table was created to show runtimes and speedup values for each system count and process setting. The table listed sequential runtimes, parallel runtimes for two and four processes, and their matching speedup factors. This table format made it easy to compare different workloads and showed the steady benefit of parallel execution. Together with the plot, the table gives numeric and visual proof of the performance gains.

```
Summary table (times in seconds):
systems  seq_time   par2_time  speedup2   par4_time  speedup4
     16     0.387      0.843      0.46x      0.103      3.74x
     32     0.929      0.402      2.31x      0.207      4.49x
     64     1.779      0.869      2.05x      0.390      4.56x
    128     3.502      1.637      2.14x      0.798      4.39x
    256     6.703      3.250      2.06x      1.417      4.73x
```

4.5 Accuracy Verification:

To check correctness, the results from sequential and parallel runs were compared using the *max_difference*() function. This step measured the largest Euclidean distance between the final states of all systems. The values were almost zero, clearly showing that parallel running gave results that fully matched the sequential baseline. This check showed that speedup was gained without losing numerical accuracy.

4.6 Interpretation of Scaling Behaviour:

The scaling behaviour seen in the experiments showed that parallel efficiency improves when the workload is larger. For small system counts, the cost of handling processes reduced the gains, but as the number of systems grew, parallel execution became more helpful. This agrees with the idea that embarrassingly parallel problems gain most when the workload is big enough to cover multiprocessing overhead. Overall, the results clearly show that parallelization is an efficient way to speed up Lorenz system simulations.

5. Performance Insights and Future Directions:

5.1 Strengths of the Approach:

The project showed clear strengths in both design and execution. Parallel work with Python's multiprocessing module gave clear speedup compared with sequential runs, showing that sharing tasks across several cores can greatly cut runtime. Accuracy was kept by checking that the parallel results matched the sequential baseline, so the gains did not harm correctness. Also, the use of reusable process pools copied real high-performance computing (HPC) practice, where resources are set up once and reused to lower overhead. This choice made the program both efficient and close to typical HPC methods.

5.2 Limitations Encountered:

Even with these strengths, some limits were found. Python's multiprocessing adds extra cost when making and handling processes, which lowers efficiency for small workloads. The tests also used only a few processes (two and four), which reduced how far the results could scale. In addition, speedup did not grow in a straight line with more processes, showing the effect of overhead and the limits of Python's multiprocessing tools. These issues show that this method is not ideal for big jobs or many cores.

5.3 Suggested Improvements:

Future improvements could deal with these limits by widening the use of parallelization. Larger process pools could be tested to study scaling behaviour more clearly in more detail. More advanced tools such as MPI (Message Passing Interface) or job schedulers could be used to cut overhead and raise efficiency on bigger systems. Also, tests on larger workloads and stronger hardware would give better understanding of how the method scales. These steps would make the project more useful in real HPC work and beyond small simulations.

6. Conclusion:

The goal of this project was to make Lorenz system simulations faster by using parallel computing methods. By building both sequential and parallel versions, the work showed that using parallel code with reusable process pools can cut runtime while still keeping accuracy. The results showed clear speedup when tasks were shared across several cores, and the gains became stronger as the workload grew. Although Python's multiprocessing adds overhead and limits scaling, the method showed the value of parallel work for embarrassingly parallel tasks.

The wider message is that parallel computing is useful for tasks that can be split into separate units, such as running many Lorenz systems. Even a small amount of parallel work can give large performance gains, so the simulations become more practical and faster to run in practice. This project shows the need to design workflows that balance accuracy, efficiency, and scaling, and how reusable process pools match real HPC practice. Overall, the results support the value of parallel computing in science, where cutting runtime without losing correctness is key for very large simulations and studies.

References:

- Yadav, J., Baig, M.A. and Mehtre, V.V. (2022) 'Review paper on comparative numerical method to solve differential equation', *International Journal of Advances in Engineering and Management (IJAEM)*, 4(11), pp. 967–970. Available at: https://ijaem.net/issue_dcp/Review%20Paper%20on%20Comparative%20Numerical%20Method%20to%20Solve%20Differential%20Equation.pdf
- Utkarsh, U., Churavy, V., Ma, Y., Besard, T., Srisuma, P., Gymnich, T., Gerlach, A.R., Edelman, A., Barbastathis, G., Braatz, R.D. and Rackauckas, C. (2024) 'Automated translation and accelerated solving of differential equations on multiple GPU platforms', *Computer Methods in Applied Mechanics and Engineering*, 419, 116591. Available at: https://web.mit.edu/braatzgroup/Utkarsh_ComputMethodsApplMechEng_2024.pdf
- Aziz, Z.A., Abdulqader, D.N., Sallow, A.B. and Omer, H.K. (2021) 'Python parallel processing and multiprocessing: A review', *Academic Journal of Nawroz University (AJNU)*, 10(3), pp. 345–350. Available at: https://pdfs.semanticscholar.org/7337/73fdf89057322ea78489912c6f769bdfbaff.pdf
- Arjona, A., Finol, G. and García López, P. (2023) 'Transparent serverless execution of Python multiprocessing applications', *Future Generation Computer Systems*, 140, pp. 436–449. Available at: https://www.sciencedirect.com/science/article/pii/S0167739X22003612
- LabEx (no date) 'How to speed up processing using multiprocessing in Python'. Available at: https://labex.io/tutorials/python-how-to-speed-up-processing-using-multiprocessing-in-python-398070